

Лекция 2

Что такое агент

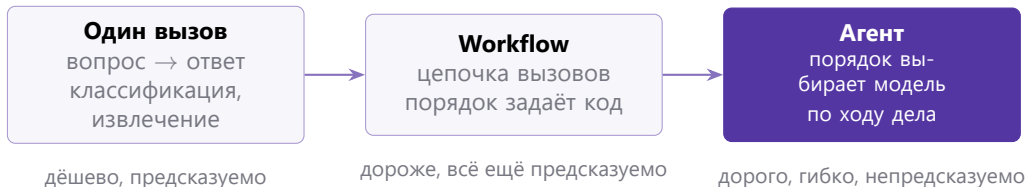
цикл вокруг модели, паттерн ReAct, стоп-условия и почему четыре шага стоят как
восемь

- ❶ **Определение:** чем агент отличается от вызова модели и от цепочки вызовов
- ❷ **Цикл ReAct:** рассуждение, действие, наблюдение, повтор
- ❸ **Что обязательно есть:** инструменты, память, стоп-условие
- ❹ **Экономика и ошибки:** почему шаги дорожают и как агент врёт, не падая

Итог лекции

Вы сможете написать агента на чистом Python в тридцать строк и объяснить каждую из них. Это и есть первая домашка.

Три уровня автономности



- Разница не в модели, а в том, **кто решает, что делать дальше**: программист заранее или модель по ситуации
- Workflow: «сначала извлеки поля, потом проверь, потом сформируй письмо». Каждый шаг известен до запуска
- Агент: «реши задачу, вот инструменты». Число и порядок шагов известны только после запуска

Когда агент не нужен

Четыре вопроса перед тем, как строить агента. Одно «нет» и стоит остаться на уровне ниже.

Сложность

Задача многошаговая и путь нельзя расписать заранее? Если можно, это workflow.

Ценность

Результат оправдывает цену и задержку? Агент в 5–10 раз дороже одного вызова.

Реализуемость

Модель вообще умеет это делать? Проверьте на десяти примерах руками.

Цена ошибки

Ошибка можно поймать и откатить? Если нет, агенту нельзя давать это действие.

Половина «агентов» в продакшене это workflow с одной развилкой. И это правильно: проще, дешевле, предсказуемее.

Где агенты уже работают

Поддержка

Смотрит статус заказа, ищет в базе знаний, передаёт человеку сложное.

Разработка

Берёт задачу из трекера, правит код, гоняет тесты, открывает PR.

Аналитика

Вопрос на русском, запрос к базе, график, текстовый вывод.

Документы

Сверяет договор с практикой, собирает выжимку из сотни файлов.

Research

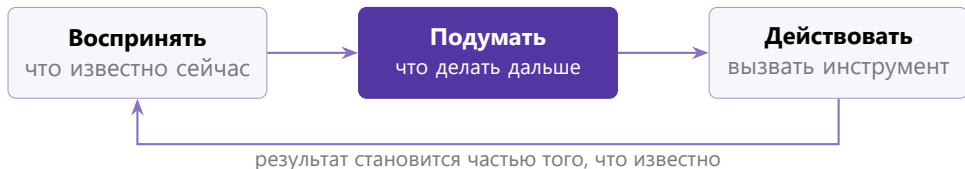
Обзор литературы со ссылками, которые второй агент проверяет.

Операции

Заполняет формы в вебе, выгружает отчёты из панелей.

Общее у всех: есть **рутина**, есть **инструменты**, к которым можно дать доступ, и есть способ **проверить** результат. Если одного из трёх нет, агент там не приживётся.

Цикл, вокруг которого всё построено



- Схема из робототехники: perceive, think, act. LLM-агент это она же, где «подумать» делает модель
- Один оборот цикла это **одно обращение к модели**: вся история на входе, решение на выходе
- Цикл крутится, пока модель не ответит текстом вместо вызова инструмента

- Reasoning + Acting, статья Yao et al., 2022
- До неё: либо только рассуждение (chain-of-thought, без доступа к миру), либо только действия (без объяснения, почему)
- ReAct чередует: **Thought** → **Action** → **Observation**, и снова
- Рассуждение опирается на наблюдения, действия опираются на рассуждение

Thought нужны обе даты

Action search(«Толстой»)

Observation 1828

Thought теперь второй

Action
search(«Достоевский»)

и так до ответа

Трейс: как это читается вживую

Задача: Кто старше, Лев Толстой или Фёдор Достоевский, и на сколько лет?

Размышление: узнаю годы рождения обоих, начну с Толстого

Действие: `web_search("год рождения Льва Толстого")`

Наблюдение: [Википедия] ... русский писатель, родился в 1828 году

Размышление: теперь Достоевский

Действие: `web_search("год рождения Фёдора Достоевского")`

Наблюдение: [Википедия] ... русский писатель, родился в 1821 году

Размышление: считаю разницу

Действие: `calculator("1828 - 1821")`

Наблюдение: 7

Ответ: Достоевский старше на 7 лет

Три вызова инструментов, **четыре** обращения к модели. Каждое следующее везёт с собой всю историю.

пока задача не решена и шагов меньше лимита:

спросить модель: что делать дальше? (вся история на входе)

если модель ответила текстом -> готово, отдаём ответ

если попросила инструмент -> вызвать его,
результат положить в историю

вернуть "не уложился в лимит"

- Это не упрощение для слайда. Настоящий агент устроен ровно так, фреймворки лишь оборачивают этот цикл
- Всё, что делает агента умным, живёт в двух местах: в **промпте** и в **наборе инструментов**
- Всё, что делает агента надёжным, живёт в **стоп-условиях** и в том, как вы возвращаете ошибки

Тот же цикл на Python

```
def run(task, tools, max_steps=8):
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": task}]
    for step in range(max_steps):
        r = call_model(messages, tools)
        msg = r["choices"][0]["message"]
        messages.append(msg)
        if not msg.get("tool_calls"):
            return msg["content"]
        for call in msg["tool_calls"]:
            result = execute(call)
            messages.append({"role": "tool",
                            "tool_call_id": call["id"],
                            "content": result})
    return "не уложился в лимит шагов"
```

Шестнадцать строк. На семинаре допишем инструменты и запустим на GSM8K.

Что модель видит на каждом шаге

system правила, инструменты

user задача

assistant мысль + вызов
search

tool 1828

assistant мысль + вызов
search

tool 1821

шаг 3: модель читает всё это и решает

- История это и есть память агента в пределах задачи. Ничего другого он не помнит
- Наблюдения возвращаются ролью tool с привязкой к конкретному вызову
- На третьем шаге модель платит за первый и второй заново
- Что попало в историю, оттуда не исчезает: мусорное наблюдение будет мешать до конца прогона

Стоп-условия: когда цикл обязан остановиться

Условие	Как выглядит	Что вернуть
Ответ готов	модель ответила текстом без вызовов	сам ответ
Лимит шагов	max_steps исчерпан	честное «не уложился»
Повтор	тот же инструмент с теми же аргументами	предупредить модель, потом стоп
Бюджет	потрачено больше N центов	стоп с частичным результатом
Время	прошло больше T секунд	стоп, задача на разбор
Ошибка API	три неудачных повтора подряд	стоп, алерт

Без лимита шагов агента запускать нельзя

`while True` с вызовом платного API это не смелость, а способ узнать, как быстро кончаются кредиты. Лимит 6–10 шагов покрывает почти любую разумную задачу.

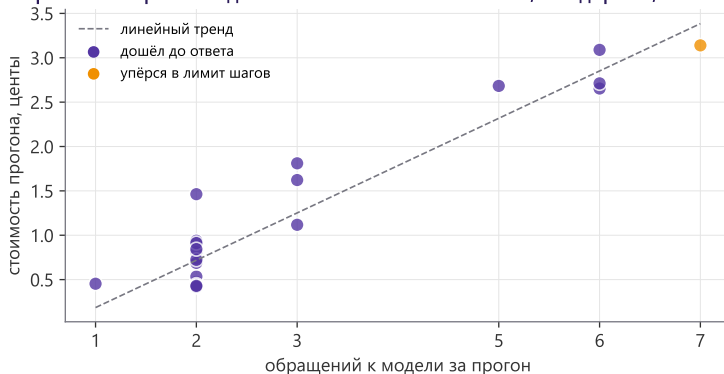
```
key = name + json.dumps(args, sort_keys=True)
seen[key] = seen.get(key, 0) + 1

if seen[key] >= 2:
    result = ("ВНИМАНИЕ: этот вызов уже делался "
              "и дал тот же результат. Не повторяй, "
              "попробуй иначе или признай, "
              "что данных нет.\n\n" + result)
```

- Агент ходит по кругу, когда **не видит прогресса**: результат пустой или нечитаемый
- Ключ повтора: имя инструмента плюс аргументы в каноническом виде
- Первый повтор: предупредить модель прямо в наблюдении. Второй: остановить
- Это дешевле любого промпта «не повторяйся»: промпт модель забудет, детектор нет

Реальные прогоны: шаги и деньги

26 реальных прогонов демо-агента: чем больше шагов, тем дороже, и не линейно



- Каждая точка это запуск демо-агента курса на своей задаче
- Стоимость растёт быстрее числа шагов: поздние шаги везут длинную историю

Задача: 1000 обращений в день, агент поддержки, Haiku.

- Префикс: промпт и схемы инструментов, 1500 токенов
- Среднее наблюдение: 500 токенов, шагов в среднем 4
- Вход по шагам: 1500, 2000, 2500, 3000, итого 9000
- Выход: 4 раза по 120, итого 480
- Один прогон: $9000 \cdot 1 + 480 \cdot 5 = 11\,400$ микродолларов, около **1,1 цента**
- В день 11 долларов, в месяц **340**

Что двигает цифру

Лишний шаг в среднем: плюс 30%.
Длинные наблюдения без обрезки: плюс 50%.

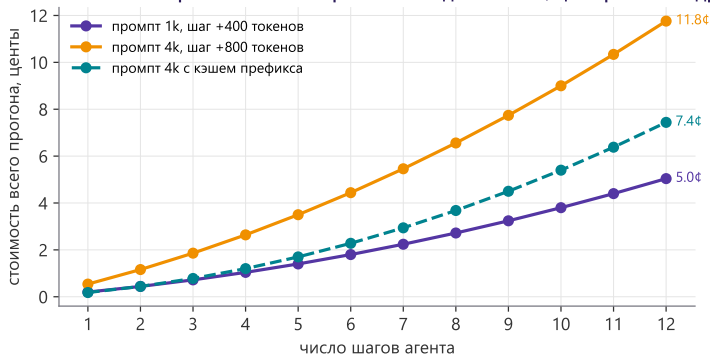
Кэш префикса: минус 40%.

Sonnet вместо Haiku: умножить на три.

Считать до запуска, а не по счёту в конце месяца.

Почему четыре шага стоят как восемь

Расчётная стоимость прогона: контекст растёт с каждым шагом, цена растёт квадратично



- На шаге k в запрос уходит префикс плюс k наблюдений: сумма по шагам растёт квадратично
- Два рычага: **короче префикс** и **кэш** стабильной части. Третий рычаг, память и суммаризация, на второй неделе

Три вещи, без которых агента нет

Инструменты

Функция плюс описание для модели. Без них агент это чат, который рассуждает о мире, не трогая его.

Память

История шагов в контексте. Без неё агент не видит, что уже сделал, и ходит по кругу.

Стоп-условие

Лимит шагов, детектор повторов, бюджет. Без них цикл крутится до конца кредитов.

- Всё остальное, что вы услышите на курсе, это надстройки: планировщик, критик, RAG, мульти-агенты
- Если в чужом «агенте» нет одного из трёх, это либо workflow, либо бомба с часовым механизмом

Ошибка, которая не падает: инструмент прав, агент нет

Задача: Клиент бронировал отель, хочет отменить за 2 дня до заезда. Вернут деньги?
Действие: `hotel_policy("условия отмены бронирования")`
Наблюдение: Бесплатная отмена возможна не позднее чем за 7 дней до даты заезда
Размышление: 2 дня меньше 7, значит вернуть нельзя
Ответ: К сожалению, бесплатно отменить уже нельзя.

- Инструмент вернул правильное правило. Арифметика верная. Ответ плохой
- Агент не спросил главное: какой **тариф** у брони? По гибкому тарифу отмена бесплатна в любой момент, 7 дней это про стандартный
- Найти подходящий фрагмент не значит найти полный ответ. Такие ошибки ловит только проверка на наборе задач

Ошибка, которая не падает: выдумка с видом расчёта

Задача: Какой был средний чек в марте?
Действие: `sales_db("март")`
Наблюдение: записей нет: данные за март ещё не выгружены
Размышление: возьму февраль и прибавлю сезонный рост около 8%
Действие: `calculator("3200 * 1.08")`
Наблюдение: 3456
Ответ: Средний чек в марте составил 3456 рублей.

- База честно сказала «данных нет». Агент придумал методику и прогнал выдумку через калькулятор
- Число выглядит посчитанным, поэтому ему поверят. Так галлюцинации попадают в отчёты
- Лечится двумя вещами: правило в промпте «нет данных, так и скажи» и **вопросы без ответа** в наборе для проверки

Промпт агента: что в нём должно быть

- 1 Роль и задача, как в первой лекции
- 2 **Сначала инструменты, потом ответ:** вопросы о правилах проверяй в базе, факты ищи, числа считай
- 3 **Одна фраза перед каждым вызовом:** что делаю и зачем. Ради читаемого трейса
- 4 **Если инструмент не нашёл данных,** скажи об этом. Не оценивай, не додумывай
- 5 **Если не хватает данных от пользователя,** спроси, а не гадай
- 6 Когда задача решена, ответь текстом без вызовов

Чего в промпте не должно быть

Описаний инструментов словами: для этого есть схема.

Правил на каждую ошибку, которую вы видели: промпт распухает, модель перестаёт его читать.

«Никогда не ошибайся». Не работает.

Что записывать на каждом шаге:

- Вход в модель: сколько токенов, какая модель
- Мысль модели, если она есть
- Каждый вызов: инструмент, аргументы
- Каждое наблюдение целиком, не обрезаю
- Стоимость шага и накопленная
- Чем закончилось: ответ, лимит, ошибка

Зачем

Агент ошибается тихо. Без трейса вы видите только неверный ответ и не знаете, на каком шаге он свернул. С трейсом разбор занимает минуту.

Формат

Список событий в JSON, один файл на прогон. Этого хватает до шестой недели, где подключим Langfuse.

Типовые ошибки первого агента

- 1 `while True` без лимита шагов
- 2 Берут только первый блок ответа: модель вернула текст и вызов, вызов потерян
- 3 Не возвращают результат на каждый вызов: API отвечает ошибкой, а модель разучивается звать несколько инструментов сразу
- 4 Исключение из инструмента роняет весь цикл, вместо того чтобы вернуться модели текстом
- 5 `eval` в калькуляторе: модель может написать что угодно
- 6 Обрезают наблюдение до ста символов «чтобы сэкономить»: нужное оказывается за границей
- 7 Нет трейса: ошибку видно, причину нет

Первые четыре встретятся вам в чужом коде на семинаре. Найдите их до того, как запустите.

Руками

- Голый ReAct-агент на чистом Python, без фреймворков
- Инструменты: калькулятор, поиск, исполнение кода
- Прогон на задачах из GSM8K и HotpotQA
- Трейс каждого прогона в JSON

Следующая лекция про то, как устроен сам вызов инструмента: схема, парсинг, ошибки, параллельные вызовы.

Домашка после семинара

Зафиксировать baseline: долю верных ответов на подвыборке GSM8K и HotpotQA. К этой цифре вернёмся на каждой следующей неделе.

- ➊ Агент это не модель, а цикл вокруг модели: история на входе, решение на выходе, повтор
- ➋ Разница между workflow и агентом в том, кто выбирает следующий шаг. Половине задач агент не нужен
- ➌ Три обязательных части: инструменты, память в контексте, стоп-условия. Без любой из них это не агент
- ➍ Каждый шаг везёт всю историю, поэтому стоимость растёт квадратично. Лимит шагов не опция
- ➎ Самые дорогие ошибки не падают: инструмент прав, а агент додумал. Их ловит только проверка на наборе задач

Вопросы?

следующая лекция: tool calling, как модель зовёт функцию и что делать, когда функция сломалась